SWP-Python 5BHWII

Lernzusammenfassung bis einschließlich KW 41
Theorie · passende Testfragen · Zusatzfragen

Quellen: **5B_SWP_python_2026.pdf**, **Python_Syntax_Kontrollstrukturen.pdf**, **Python_Testfragen_Sammlung** (nur Fragen zum Stoff bis KW 41).

Teil 1 — Stoffübersicht bis KW 41

Rahmenbedingungen (Kurz)

- Drei oder vier Tests; jeder Test muss positiv sein (sonst P5).
- Mitarbeit 30 % · Tests 70 %; 1. HJ 45 % · 2. HJ 55 %.
- Mitarbeit startet bei Befriedigend; Plus –0,25 · Minus +0,5.
- HUs gehören in ein eigenes Git-Repo; Folien: github.com/htlinnrain/python
- Stoff allgemein: Python-Theorie + Aufgaben → verkettete Listen → 2. HJ Projekt.

KW 38 — Warum Grundlagen trotz KI?

Kernaussage: Wer den Code nicht versteht, steuert nicht – er fliegt blind.

- **Debugging & Verifikation:** KI liefert schnell laufenden Code, baut aber oft subtile Logikfehler, Race Conditions oder Sicherheitslücken ein. Ohne Grundlagen, Datenstrukturen und Systemverständnis kann man KI-Code weder prüfen noch reparieren.
- **Pilot-Analogie:** Ein Jet fliegt oft auf Autopilot – der Pilot muss das Flugzeug trotzdem im Detail verstehen, um bei Ausfällen die Kontrolle zu übernehmen.
- **Programmieren = Problemübersetzung**, nicht Syntax-Tippen. Architektur, Schnittstellen, Skalierung und echte Problemstellung sind die eigentliche Ingenieursleistung.
- **Präzise Anforderungen:** Gute Prompts brauchen technische Durchdringung. Vage Eingaben → unbrauchbarer Code.
- **Haftung:** Bei Fehlern in kritischen Systemen (Medizin, Automotive, Finanzen) haftet der Entwickler, nicht das Sprachmodell.
- **Rollenwandel:** KI ersetzt mechanische Tipparbeit. Entwickler mit soliden Informatikgrundlagen + KI als Werkzeug werden produktiver, nicht arbeitslos.

KW 39 — Einführung & Python-Grundlagen

Praktischer Einstieg

- Einführung, Folien via Git, eigenes Repo für SWP-Python anlegen.
- Python installieren, Hallo-Welt programmieren.
- Lernressourcen: learnpython.org, programiz.com/python-programming.
- KI als Recherchehilfe erlaubt – Nutzung in Git protokollieren.

Python 2 vs. Python 3

Python 3 hat die Abwärtskompatibilität zu Python 2 bewusst gebrochen (z. B. `print` als Funktion, Division `/` liefert float, Unicode-Strings). Heute arbeitet man praktisch nur noch mit Python 3.

Iterativ vs. rekursiv

Iterativ: Lösung mit Schleifen (for/while), Zustand in Variablen. **Rekursiv:** Funktion ruft sich selbst auf, braucht Abbruchbedingung. Rekursion ist oft eleganter bei Baum-/Teile-und-Herrsche-Problemen, braucht aber Stack und kann bei großer Tiefe overflowen.

```
def fak_iter(n):
    r = 1
    for i in range(2, n + 1):
        r *= i
    return r

def fak_rek(n):
    if n <= 1:
        return 1
    return n * fak_rek(n - 1)
```

Prozesse vs. Threads

- **Prozess:** eigenständiges Programm mit eigenem Adressraum (Speicher). Kommunikation nur über IPC (Pipes, Sockets, …). Isoliert, aber schwerer.
- **Thread:** Ausführungspfad innerhalb eines Prozesses; Threads teilen sich Speicher. Leichtgewichtiger, aber Race Conditions möglich.
- In CPython begrenzt der GIL oft echte Parallelität bei CPU-lastigem Code (Threads gut für I/O; Prozesse für CPU).

Typisierung & Type Hints

- Python ist **dynamisch typisiert**: Variablen haben keinen festen Typ zur Compile-Zeit; der Typ hängt am Objekt zur Laufzeit.
- Gegenstück: **statische Typisierung** (z. B. Java) – Typen werden vor dem Lauf geprüft.
- **Type Hints** (ab 3.5+): optionale Annotationen für Lesbarkeit und statische Checker (mypy, pyright) – zur Laufzeit oft ignoriert.

```
def addiere(a: int, b: int) -> int:
    return a + b

namen: list[str] = ["Ana", "Ben"]
```

Paradigmen, Interpreter, Speicher

- Python unterstützt **funktional** und **objektorientiert**.
- **Interpretersprachen** (Python, JS): Quellcode wird zur Laufzeit interpretiert/JIT. **Compilersprachen** (C, C++, oft Java→Bytecode): Übersetzung vor dem Lauf.
- **Instanz:** konkretes Objekt im Speicher. **Referenz:** Name/Zeiger auf diese Instanz. **Referenzzähler:** zählt, wie viele Referenzen auf ein Objekt zeigen. **Garbage Collection:** freigibt Speicher, wenn nichts mehr darauf zeigt (+ Zyklus-GC). **del**: entfernt einen Namen/Referenz (Objekt erst weg, wenn Zähler 0).

Syntax & Kontrollstrukturen

Blöcke über **Einrückung** (4 Leerzeichen, keine Tabs) und Doppelpunkt. Logik: `and / or / not` (nicht && || !). `True / False / None` (groß).

```
# if / elif / else
note = 78
if note >= 90:
    print("sehr gut")
elif note >= 75:
    print("gut")
else:
    print("nicht genügend")

# bedingter Ausdruck (ternär)
status = "ok" if note >= 50 else "fail"

# match/case (viele Fälle – analog switch)
match note // 10:
    case 10 | 9:
        print("1")
    case 8 | 7:
```

```
            print("2")
        case _:
            print("andere")

# Schleifen
for i in range(5):          # 0..4
    print(i)
while n > 0:
    n -= 1                  # kein ++/--

# break / continue / pass
for i in range(10):
    if i == 3:
        continue
    if i == 7:
        break
    print(i)
def noch_offen():
    pass                    # leerer Block

# try / except / else / finally
try:
    ergebnis = a / b
except ZeroDivisionError:
    print("Division durch 0")
else:
    print("ok")             # nur ohne Exception
finally:
    print("fertig")         # immer
```

- **if-elif-else:** gut für Bereiche/Bedingungen. **match/case:** gut, wenn viele konkrete Werte/Muster eines Inhalts geprüft werden.
- Schleifen-**else**: läuft nur, wenn die Schleife *nicht* per break endet.
- Exceptions: `raise ValueError("...")`

KW 40 — Arrays, Listen, Dictionaries, Lotto

Wiederholung + Vergleich Assembler/C++ (Abbildung in den Folien): hohe Abstraktion in Python vs. maschinennahe Sprachen.

Datenstrukturen

- **list** (Python): dynamisch, gemischte Typen möglich, ähnlich Java `ArrayList`.
- **array** (Modul `array`) / **numpy.ndarray**: echte Arrays – homogener Typ, speichereffizient, schneller bei Zahlenrechnungen.
- Java-Arrays: feste Länge, fester Typ. Python-Listen: flexibel.
- **dict**: Zuordnung Schlüssel → Wert (Hash-Map). Ideal für Zähler/Statistik.
- Binary Search (programiz): Algorithmus + Python-Syntax der Schleifen/Indizes.

Aufgabe: Lottoziehung

- Methode, die 6 Zahlen aus 1–45 zieht und ausgibt.
- Wichtig: **keine Zahl doppelt**. Algorithmus so, dass der RNG bei 45 gezogenen Zahlen nur 45-mal aufgerufen wird (z. B. Fisher-Yates / Liste mischen und erste 6 nehmen – nicht „solange neu würfeln bis unique“).
- Statistik: 1000 / 10 000 / 100 000 Ziehungen; Dictionary zählt, wie oft jede Zahl vorkommt; nach jeder Ziehung Zähler erhöhen.

```
import random

def lottoziehung() -> list[int]:
    zahlen = list(range(1, 46))
```

```
        random.shuffle(zahlen)       # RNG-Aufrufe begrenzt
        gezogen = sorted(zahlen[:6])
        return gezogen

    def statistik(n: int) -> dict[int, int]:
        zaehler = {i: 0 for i in range(1, 46)}
        for _ in range(n):
            for z in lottoziehung():
                zaehler[z] += 1
        return zaehler
```

KW 41 — Datentypen, Division, Stil, Schlüsselwörter

- Type-Hints, Data-Validation, Datentypen vertiefen (int, float, str, list, dict).
- **Division:** `/` → float; `//` → Floor Division (abrunden Richtung $-\infty$); `%` Rest; `divmod(a,b)`.
- Strings: mit + konkatenieren; immutable (neu erzeugt).
- Variablennamen sind **case-sensitiv** (`Name` ≠ `name`).
- Schlüsselwörter reserviert (`if, for, def, class, True, None, …`) – siehe Rheinwerk OpenBook.
- Logische Ausdrücke: `and / or / not`, Vergleiche `< <= == != >= >`.
- Dateien enden auf `*.py`.
- Shebang (UNIX): `#!/usr/bin/env python3`
- Stil: keine Tabs, 4 Spaces; lange Zeilen max. ~80 Zeichen, Umbruch mit Backslash `\`.
- Kommentare: `# …` oder Docstrings `"""…"""`; Platzhalter: `pass`.

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(-7 // 2)  # -4  (Floor!)
print(7 % 2)    # 1

#!/usr/bin/env python3
# kommentar
def hilfe():
    """Kurze Doku."""
    pass
```

Schnellvergleich Python vs. Java (bis KW 41)

Thema	Python	Java
Blöcke	Einrückung	{ }
Typen	dynamisch + optionale Hints	statisch, Pflicht
Logik	and / or / not	&& || !
Boolean/Null	True False None	true false null
else if	elif / match	else if / switch
for	über Iterable / range	C-Style oder for-each
++ / --	gibt es nicht	ja
Exceptions	except (+ else)	catch
Listen/Arrays	list flexibel; array/np fest	Array fest; ArrayList dynamisch

Teil 2 — Fragen aus der Testfragen-Sammlung

Nur Fragen, die zum Stoff **bis einschließlich KW 41** passen. Spätere Themen (List Comprehensions, mutable/immutable Vertiefung, OOP, Lambda, unittest, verkettete Listen, …) sind bewusst weggelassen. Musterantworten zum Lernen – beim Test können Formulierungen abweichen.

F1. Erklären Sie Instanz, Referenz, Referenzzähler, Garbage Collection und del.

Antwort: Eine **Instanz** ist ein konkretes Objekt im Speicher. Eine **Referenz** ist ein Name/Zeiger darauf. Der **Referenzzähler** zählt, wie viele Referenzen auf das Objekt zeigen. Die **Garbage Collection** gibt Speicher frei, wenn nichts mehr darauf zeigt (plus Erkennung von Referenzzyklen). **del name** entfernt die Referenz; das Objekt verschwindet erst, wenn der Zähler 0 ist.

F2. Was sind Type Hints?

Antwort: Optionale Typannotationen an Parametern, Rückgaben und Variablen (z. B. `def f(x: int) -> str:`). Sie dokumentieren Erwartungen und ermöglichen statische Prüfung (mypy), ändern aber zur Laufzeit meist nichts am dynamischen Verhalten.

F3. Erklären Sie den Unterschied zwischen Compiler- und Interpretersprachen.

Antwort: Compiler: Quellcode wird vor dem Ausführen in Maschinencode/Bytecode übersetzt (C, C++, Java→Bytecode). **Interpreter:** Quellcode wird zur Laufzeit zeilen-/blockweise ausgeführt (Python, oft JS). Vorteil Interpreter: schneller Start/Portabilität; Vorteil Compiler: oft höhere Laufzeitleistung.

F4. Unterschiede zwischen Java und Assembler.

Antwort: Assembler ist maschinennah (Register, Befehle der CPU), kaum Abstraktion. Java ist hochsprachlich: Objekte, GC, JVM-Bytecode, Plattformunabhängigkeit. Assembler: maximale Kontrolle/Leistung, wenig Portabilität; Java: Produktivität und Sicherheit auf Kosten von Abstraktionsschichten.

F5. Unterschied if-elif-else und match case.

Antwort: if-elif-else: beliebige Bedingungen/Bereiche. **match/case:** Musterabgleich auf einen Wert (viele konkrete Fälle, Zerlegung von Strukturen). Für „viele Möglichkeiten eines Inhalts“ oft match/case klarer.

F6. Was sind Kontrollstrukturen? Welche eignet sich, um viele Möglichkeiten eines Inhalts zu überprüfen?

Antwort: Kontrollstrukturen steuern den Programmfluss: Verzweigungen (if/match), Schleifen (for/while), Sprünge (break/continue), Ausnahmebehandlung (try/except). Für viele Möglichkeiten eines Inhalts: **match/case** (früher oft lange if-elif-Ketten).

F7. Unterschied zwischen iterativer und rekursiver Programmierung.

Antwort: Iterativ: Schleifen + Zustandsvariablen. Rekursiv: Funktion ruft sich selbst mit kleinerem Teilproblem auf, braucht Basisfall. Rekursion oft eleganter bei Bäumen; Iteration oft speicher-/stackschonender.

F8. Nennen Sie drei fachliche Unterschiede zwischen Java und Python.

Antwort: 1) Typisierung: Java statisch / Python dynamisch (+ optionale Hints). 2) Blöcke: Java { } / Python Einrückung. 3) Ausführung: Java Compiler→Bytecode+JVM / Python Interpreter. (Weitere: kein ++ in Python; True/None vs. true/null; list vs. Array/ArrayList.)

F9. Prozesse und Threads allgemein in Betriebssystemen.

Antwort: Ein **Prozess** ist ein laufendes Programm mit eigenem Adressraum. **Threads** sind parallele Ausführungspfade im selben Prozess und teilen sich Speicher. Prozesse: Isolation; Threads: leicht, aber Sync nötig.

F10. Unterschied zwischen Arrays in Python und Java.

Antwort: Java-Arrays: feste Länge, einheitlicher Typ. In Python ist die Standard-Sequenz die **list** (dynamisch, gemischte Typen). „Echte“ Arrays gibt es über das Modul `array` oder NumPy – homogener Typ, fester/effizienter Speicher.

F11. Unterschied zwischen array und list in Python sowie die Vorteile von Arrays.

Antwort: list: flexibel, beliebig typisiert, allgemein. **array/np.array:** ein Elementtyp, dichter Speicher, schneller bei großen Zahlenmengen und numerischen Operationen.

F12. Nennen Sie drei Vorteile von Python gegenüber C-ähnlichen Sprachen.

Antwort: 1) Hohe Produktivität / knappe Syntax. 2) Dynamische Typen + großer Standard-/Library-Ökosystem. 3) Automatische Speicherverwaltung (GC). (Auch: plattformübergreifend, gut zum Prototyping/Scripting.)

F13. Drei Gemeinsamkeiten zwischen Java und Python.

Antwort: Beide: objektorientiert möglich; automatische Garbage Collection; Exceptions zur Fehlerbehandlung; große Standardbibliotheken; plattformübergreifend (JVM bzw. Interpreter).

F14. Erklären Sie die pass-Anweisung in Python.

Antwort: `pass` ist eine No-Op-Anweisung: sie macht nichts und hält einen Block syntaktisch gültig (leere Funktion/Klasse/if-Zweig), wo Java leere { } hätte.

F15. Unterschied im Umgang mit Datentypen zwischen Java und Python, einschließlich Type Hints.

Antwort: Java verlangt Deklaration und prüft Typen zur Compile-Zeit. Python bindet Typen an Objekte zur Laufzeit. Type Hints bringen optionale statische Hinweise ohne das dynamische Modell abzuschaffen.

F16. Floor Division in Python 3.

Antwort: `//` teilt und rundet Richtung $-\infty$ (Floor). Beispiele: `7 // 2 == 3`, `-7 // 2 == -4`. Dagegen liefert `/` immer float (`7 / 2 == 3.5`).

F17. Garbage Collection allgemein.

Antwort: Automatische Freigabe nicht mehr erreichbarer Objekte. In CPython vor allem Referenzzählung; zusätzlich ein Zyklus-Collector für zirkuläre Referenzen.

F18. Beispiel für den ternären Operator / bedingten Ausdruck.

Antwort: Konventionell: `if x > 0: s = "pos"\nelse: s = "neg"`. Ternär: `s = "pos" if x > 0 else "neg"`.

F19. Erläutern Sie den ternären Operator und geben Sie ein konventionelles Beispiel sowie die Umsetzung als ternären Operator an.

Antwort: Kurzform von if/else für Ausdrücke. Konventionell: if note >= 50: status = "ok" else: status = "fail". Ternär: status = "ok" if note >= 50 else "fail".

F20. Exceptions anhand eines kompletten Code-Beispiels erklären (alle Exception-Teile).

Antwort: try: riskanter Code · except Typ as e: Behandlung · else: nur ohne Fehler · finally: immer (Aufräumen). Zusätzlich raise zum Auslösen.

F21. Arten von Exceptions erklären; vier Exception-Kombis demonstrieren.

Antwort: Strategien: LBYL (vorher prüfen) vs. EAFP (try/except). Fehlerarten: neu erzeugt vs. „hochgeblubbert“; behebbar vs. nicht behebbar. Vier Kombis: (a) neuer + behebbar → Default setzen; (b) hochgeblubbert + behebbar → except und Alternative; (c) neuer + nicht behebbar → raise; (d) hochgeblubbert + nicht behebbar → nicht schlucken, nach oben lassen bzw. am Einstieg loggen und exit.

Teil 3 — Zusatzfragen (Stoff bis KW 41)

Weitere Fragen, die man mit dem behandelten Stoff beantworten können sollte.

Z1. Warum reicht „der Code läuft“ laut KW 38 nicht als Qualitätskriterium?

Antwort: Weil KI/schnell geschriebener Code subtile Logik- und Sicherheitsfehler enthalten kann. Nur wer Grundlagen beherrscht, kann verifizieren und reparieren.

Z2. Was bedeutet die Pilot-Analogie im Kontext von KI-generiertem Code?

Antwort: Autopilot (KI) erledigt Routine; der Pilot (Entwickler) muss System und Grundlagen kennen, um bei Ausfällen die Kontrolle zu übernehmen und zu haften.

Z3. Nennen Sie zwei konkrete Python-2→3-Brüche.

Antwort: print ist Funktion; / bei ints liefert float; str ist Unicode; viele Standardbibliotheks-Umbenennungen.

Z4. Wann würden Sie Rekursion vermeiden?

Antwort: Bei sehr großer Rekursionstiefe (Stack Overflow), engen Performance-/Speicher-Anforderungen oder wenn die iterative Form klarer ist.

Z5. Warum sind Threads im selben Prozess riskanter als getrennte Prozesse?

Antwort: Gemeinsamer Speicher → Race Conditions, wenn Synchronisation fehlt. Prozesse sind isolierter.

Z6. Ändern Type Hints das Laufzeitverhalten von Python?

Antwort: In der Regel nein – sie sind Hinweise für Menschen und Checker. (Ausnahmen: Tools/Libraries, die Annotations aktiv auswerten, z. B. zur Validierung.)

Z7. Was passiert bei del x, wenn noch eine andere Variable auf dasselbe Objekt zeigt?

Antwort: Nur der Name x verschwindet; der Referenzzähler sinkt um 1. Das Objekt bleibt, solange andere Referenzen existieren.

Z8. Schreiben Sie ein match/case für Wochentage (1–7) mit Default.

Antwort: match tag: case 1: … case 7: … case _: print("ungültig")

Z9. Was macht for-else in Python?

Antwort: Der else-Zweig einer for-Schleife läuft, wenn die Schleife **nicht** mit break beendet wurde (typisch: „kein Treffer gefunden“).

Z10. Warum ist „solange random, bis 6 unique Zahlen“ für die Lotto-Aufgabe ungeeignet?

Antwort: Weil die Vorgabe lautet: bei 45 gezogenen Zahlen den RNG nur 45-mal nutzen. Wiederholtes Nachwürfeln verletzt das und kann theoretisch endlos laufen.

Z11. Wie speichern Sie in der Lotto-Statistik die Häufigkeiten am sinnvollsten?

Antwort: Als dict mit Schlüsseln 1…45 und Zählwerten; nach jeder Ziehung die sechs gezogenen Zahlen inkrementieren.

Z12. Was ist der Unterschied zwischen / und // bei negativen Zahlen?

Antwort: / liefert float-Ergebnis der Division. // floored Richtung −∞, daher -7 // 2 == -4, nicht -3.

Z13. Warum sind Variablen in Python case-sensitiv – Beispiel?

Antwort: Score und score sind verschiedene Namen; Zuweisung an einen ändert den anderen nicht.

Z14. Wozu dient die Shebang-Zeile?

Antwort: Unter UNIX sagt sie dem System, mit welchem Interpreter die Datei ausgeführt werden soll, z. B. #!/usr/bin/env python3.

Z15. Welche Einrückungsregel gilt laut Stoff / Stil?

Antwort: Keine Tabs, genau 4 Leerzeichen; Zeilen möglichst ≤ 80 Zeichen, Umbruch mit \.

Z16. Wann verwenden Sie pass?

Antwort: Als Platzhalter, wenn Syntax einen Block verlangt, aber noch keine Logik existiert (Stub-Funktion, leerer except vorübergehend – vorsichtig!).

Z17. Nennen Sie drei Python-Schlüsselwörter.

Antwort: z. B. if, else, elif, for, while, def, class, return, import, try, except, True, False, None, pass, break, continue, …

Z18. Wie lösen Sie eine Exception selbst aus?

Antwort: raise ValueError("Beschreibung") – oder einen anderen Exception-Typ.

Z19. Was ist der Unterschied zwischen list und dict in einem Satz?

Antwort: list: geordnete Sequenz indexiert ab 0; dict: Abbildung Schlüssel → Wert.

Z20. Warum eignet sich ein Dictionary besonders für die Lotto-Statistik?

Antwort: Direkter Zugriff „Zahl → Häufigkeit“ in amortisiert O(1), ohne jedes Mal eine Liste zu durchsuchen.

Viel Erfolg beim Lernen. Beim echten Test können auch andere/weitere Fragen kommen.